Live Show Radar — Components Map

An AI-powered live-show alert system for one user in Bengaluru. It monitors concert and festival announcements across India for every artist he follows and alerts him when one appears, with early-bird ticket windows called out first. Every AI step runs on Google Gemini Flash through the official @google/genai SDK; no other model is used anywhere. Live at live-show-radar-pink.vercel.app

Actor / Platform	Trigger	Input	Context	Processing	AI	Output
1. Scheduler **GitHub Actions + Vercel Cron**	Cron: every 30 min; Vercel daily 02:30 UTC (08:00 IST); manual dispatch	None	APP_URL and CRON_SECRET as repository secrets; concurrency group prevents overlap	curl POST to /api/scan with a bearer token; fails the step on any non-2xx	—	A scan run; JSON summary written to the Actions run summary
2. Scan orchestrator **Next.js route on Vercel**	GET or POST /api/scan	trigger type; Authorization: Bearer CRON_SECRET or an admin session	scan_runs row; artists due by next_check_at; 300 s function ceiling	Opens a run, picks a bounded batch of 15 artists, fans out sources at concurrency 5 with 10 s timeouts, then closes the run with counts and per-source errors	—	scan_runs row; JSON summary of artists, items, shows, alerts and errors
3. Artist sync **Spotify Web API**	Daily Vercel cron, or Sync Spotify in the admin UI	GET /v1/me/following?type=artist&limit;=50, cursor-paginated	OAuth tokens stored server-side and refreshed; scope user-follow-read	Inserts new follows; reactivates re-followed artists; deactivates unfollowed ones. Never touches manual artists	—	artists rows with source 'spotify', names and images
4. Artist intake **Search, paste, CSV**	Admin types in the header search, or pastes/imports CSV	Free-text query, or one name per line with optional aliases	Spotify catalogue via app-level token; existing list used to flag duplicates	Debounced catalogue search with a conservative name match; quote-aware CSV parsing; conflicts ignored	—	artists rows with source 'manual', with Spotify artwork
5. News fetcher **Google News RSS**	Every scan, per artist and per alias	Query: "<artist>" (India OR Bengaluru OR …) (concert OR tour OR tickets …) when:14d	India feed: hl=en-IN, gl=IN, ceid=IN:en. No API key	Fetches and parses XML with fast-xml-parser; normalises to {source, url, title, snippet, published_at}	—	Normalised items, source 'google_news'
6. Ticketing search **Tavily Search API**	Platform sweep at most every 6 h; one query a day per priority artist	Sweep queries, or '<artist> India tour tickets'	search_depth basic, include_answer false, country india; include_domains for District, BookMyShow, Skillbox, SortMyScene, HighApe; free tier 1,000 credits/month	POST /search, normalises results; caps enforced in code against the database	—	Normalised items, source 'tavily'
7. Grounded search **Gemini google_search tool**	Starred artists only, when ENABLE_GEMINI_GROUNDING is true (off by default)	Natural-language prompt asking only about India dates and ticket windows	Paid tier; cannot be combined with structured output	Harvests the URLs the model grounded on; the model's prose is kept only as a snippet, never trusted as fact	Gemini Flash — **ground**	Normalised items, source 'gemini_search'; an ai_calls row
8. Filter **Deduplication**	After every fetch	All normalised items from this scan	url_hash unique index across all previous scans; 30-day age window	Canonicalises the URL (strips utm/fbclid/oc, sorts params, drops www and fragments), SHA-256 hashes it, drops duplicates and stale items	—	source_items rows; only genuinely new items continue
9. Show extractor **Gemini Task 1**	Any artist with unprocessed items; one call per artist, plus one for sweep items	Artist names with aliases, plus up to 25 unprocessed items	Appendix B JSON schema; today's date; full artist list for sweep items	Structured JSON output validated with Zod, one retry on invalid output; reports near misses honestly instead of dropping them	Gemini Flash — **extract**	Show candidates with evidence quotes and a source URL
10. Triage and matcher **Rules in code**	Immediately after extraction	Show candidates	Keep rules: is_show_announcement, country = India, confidence ≥ 0.7, date today or later	Builds dedup_key = artist_id | normalised city | event_date, normalising Bangalore, Gurgaon, Bombay and New Delhi; refuses invalid calendar dates	—	Accepted candidates, or a typed rejection reason
11. Rejection log	Whenever triage refuses a candidate	The refused candidate and its reason	Six reasons: not a show, not India, low confidence, past event, no city, unknown artist	Stores the claimed artist, country, city, confidence, evidence quote and source URL	—	rejections rows, shown on /filtered and filterable by reason
12. Reconciler **Gemini Task 2**	A candidate whose dedup_key already exists	The stored show and the new candidate	A null from the model means unreported, never cleared	Decides same_show, lists changed fields and the single material change; merges only fields the candidate actually states	Gemini Flash — **reconcile**	Updated show row; a material-change flag when it is worth an alert
13. Prioritiser **Rules in code**	Every new show and every material change	City, confidence, early-bird status and deadline	P0 Bengaluru with early bird open or imminent; P1 Bengaluru or a deadline within 72 h; P2 elsewhere in India; P3 confidence below 0.8	Deterministic. The model never sets priority — it only explains it	—	A priority of P0 to P3 stored on the show
14. Alert writer **Gemini Task 3**	Every new show, material change and early-bird reminder	The show, home city, the priority already decided, and the tag	Telegram HTML only; 600-character budget measured on visible text, not markup	Writes headline, message, action_by and the reason for the priority; a P3 never reaches this step	Gemini Flash — **alert**	Alert text ready to send
15. Notifier **Telegram Bot API, Resend**	After the alert is written	Alert HTML and the chat ID or email address	alert_key = show_id | alert_type | channel | change_ref, unique in Postgres	The row is claimed BEFORE sending, so the unique constraint is the lock: two overlapping scans cannot both send. Email is claimed separately	—	Telegram message, optional email, and an alerts row with delivery status
16. Early-bird reminder	Every scan	Shows whose early-bird deadline falls within 24 h	Only while the status is still 'open'; the deadline itself is the change_ref	Finds due shows and routes them through the alert writer; a moved deadline may alert again, an unchanged one cannot	Gemini Flash — **alert**	One reminder per deadline
17. Artist suggestions **Gemini**	Admin presses Suggest artists	Every followed artist, starred ones weighted first	Spotify's own recommendations API is closed to apps created after Nov 2024, so this uses the same model as the pipeline	Suggests similar artists likely to play India, excluding anyone already followed; images looked up in the Spotify catalogue	Gemini Flash — **suggest**	artist_suggestions rows with Add and Dismiss actions
18. Database **Postgres on Neon**	Every stage	Reads and writes from the whole pipeline	Drizzle ORM with SQL migrations; all timestamps stored in UTC	Tables: users, artists, scan_runs, source_items, shows, alerts, ai_calls, artist_suggestions, rejections. Uniqueness enforced on url_hash, dedup_key and alert_key	—	The single source of truth for every page
19. Web app **Next.js pages**	A request from the browser	Query parameters for city, priority, month and reason	Every page renders live rows; times displayed in Asia/Kolkata	Dashboard, calendar, artists, filtered-out, alerts, scan log and how-it-works, all read from Postgres on each request	—	The rendered interface
20. Access control	Any page request; any admin action	Admin password, or a bearer token for the schedulers	Session cookie stores a hash of the password, never the password	Middleware gates every page and redirects to /admin; /api/scan keeps its own bearer check so a scheduler fails loudly instead of following a redirect	—	An admin session, or a redirect to sign in

Notes

Live data only. No show is hard-coded or mocked. The single piece of seeded data is the starting artist list; every show on the dashboard was found at runtime.

Division of labour. The model judges language — is this a real show, by this artist, in India. Code decides consequences — priority, deduplication, scheduling and whether to send. Priority is never set by the model.

Failure is contained. A failure in one source, one artist or one AI call is recorded against the scan run and never fails the run. Invalid model output is retried once, then logged and skipped.

Observability. Every Gemini call is logged to ai_calls with task, model, latency and token counts. Every refused candidate is kept in rejections with the quote it was judged on.